SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-019: Student Profile Editing

Field	Details
Change Request ID	CR-019
Change Request Title	Student Profile Editing
Student Name	MH Anush
Student ID	JFICT2416
Implementation Status	Completed
Date	29 October 2026

1. Change Request Summary

1.1 Business Requirement

Currently, the Student Management System allows an authenticated student to view their own profile through the Student Dashboard, but there is no student-facing facility to edit the displayed profile information. The system should allow a student to update their own email address and phone number without administrator assistance.

To improve usability, account autonomy, and operational efficiency, CR-019 introduces a self-service profile editing capability directly within the student dashboard. Students who are actively logged into their portal can securely update their own email address and phone number while enforcing strict client-side and server-side validation, duplicate email collision prevention, and role-based access boundaries.

1.2 Existing Behaviour

Before implementation:

- `StudentDashboardPage.jsx` loads the authenticated student's profile using `GET /api/students/me`.
- The My Profile section displays registration number, full name, email, phone number, and member-since date.
- The displayed profile information is read-only; no Edit Profile button or student update form is available.
- The existing `PUT /api/students/:id` update route is protected for ADMIN users and is intended for administrator updates.
- Student authentication is handled by the existing JWT authentication middleware.
- Students requiring an email or phone update had to rely on administrator intervention.

1.3 Requested Change

Add a student-facing Edit Profile facility to the My Profile section. The implementation must:

- Add an Edit Profile button to `StudentDashboardPage.jsx`.
- Provide an accessible form for editing the student's email address and phone number.
- Pre-populate the form with the student's current values.
- Validate email and phone number before submission on both client and server.
- Add a student-authorized profile update API, preferably `PUT /api/students/me`, rather than exposing a student_id for self-update.
- Identify the student from the authenticated JWT `userId` (`req.user.userId`).
- Update only the authenticated student's existing record in the database.
- Return a clear success or error response from the backend.
- Refresh the displayed profile information immediately after a successful update.
- Preserve the existing `GET /api/students/me` profile retrieval behaviour.
- Preserve existing administrator CRUD and authorization behaviour.

2. Implementation

2.1 Implementation Summary

The CR-019 requirements were implemented systematically across the database access layer, backend API controller and routes, and frontend student portal views without requiring any database schema migration. The required email and phone_number fields already exist on the students table.

On the backend, a secure `updateStudentProfile(userId, email, phoneNumber)` model method was added to `studentModel.js`, executing a scoped parameterized SQL update bounded strictly by `user_id`. Server-side validation was implemented in `validation.js` via

`validateStudentProfileData`, enforcing RFC-compliant email formatting and 10-digit numeric phone rules. A dedicated `updateMyProfile` controller was created in `studentController.js` and exposed via `PUT /api/students/me` in `studentRoutes.js` protected by `authenticate` JWT middleware. The controller extracts `req.user.userId`, validates payload data, checks for duplicate email collisions across other student records (HTTP 409), executes the update, and returns the refreshed profile.

On the frontend, `StudentDashboardPage.jsx` was enhanced with profile editing state management, an Edit Profile action button, an accessible pre-populated edit form, real-time client validation, Save Changes and Cancel controls, loading indicators, and responsive styling in `studentDashboard.css` supporting both Light and Dark modes.

2.2 Affected Files

File	Change	Purpose
backend/src/models/studentModel.js	Modified	Implemented updateStudentProfile(userId, email, phoneNumber) executing scoped SQL update query bounded by authenticated user_id.
backend/src/utils/validation.js	Modified	Added validateStudentProfileData(data) validating presence, email format regex, numeric digits, and exact 10-digit phone length.
backend/src/controllers/studentController.js	Modified	Implemented updateMyProfile controller to validate payload, enforce duplicate email uniqueness across students, update record, and return updated profile.
backend/src/routes/studentRoutes.js	Modified	Registered PUT /me route protected by authenticate JWT middleware ahead of /:id wildcard routes.
frontend/src/pages/StudentDashboardPage.jsx	Modified	Added Edit Profile button, pre-populated form state, client validation, Save Changes/Cancel buttons, and refreshed profile view.
frontend/src/styles/studentDashboard.css	Modified	Added styles for .edit-profile-btn, .profile-edit-form, .profile-edit-info-banner, inputs, error states, and dark mode variants.

Additional files: None / [List any additional files if modified]

2.3 Technical Changes

Backend Model & Scoped SQL Update

- The `students` table contains `user_id` as a unique foreign key referencing `users(user_id)`.
- Implemented parameterized method `updateStudentProfile(userId, email, phoneNumber)` in `studentModel.js`:
- `UPDATE students SET email = ?, phone_number = ? WHERE user_id = ?`
- Deriving the student identity strictly from the authenticated JWT session token prevents any unauthorized cross-student modifications.

Server-Side Validation & Duplicate Email Handling

- Added `validateStudentProfileData` in `validation.js` enforcing email presence and regex format `^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`.
- Enforced numeric digits `^\d+$` and exact 10-digit length for `phone_number`.
- Controller verifies email uniqueness across other accounts: `SELECT student_id FROM students WHERE email = ? AND user_id != ? LIMIT 1`; returns HTTP 409 Conflict if taken.

Backend Route Registration & JWT Authentication

- Registered `router.put('/me', authenticate, updateMyProfile)` in `studentRoutes.js`.
- Route is declared prior to `router.put('/:id', ...)` to prevent route parameter collision in Express.
- The administrator update endpoint `PUT /api/students/:id` remains strictly protected by `authorizeRoles('ADMIN')`.

Frontend Component Architecture & Reactive State Management

- Added state hooks in `StudentDashboardPage.jsx`: `isEditingProfile`, `profileFormData`, `profileValidationErrors`, and `profileUpdating`.
- Rendered Edit Profile button in section header row with dynamic toggle logic.
- Constructed accessible form displaying immutable Registration No and Full Name alongside editable fields.
- Implemented client-side validation providing real-time visual feedback and field error clearance on user input.
- Integrated dismissible success and error alert banners matching the dashboard theme.

2.4 Implementation Logic

1. Student navigates to Student Portal (/student-dashboard); profile is loaded via GET /api/students/me.

2. Student clicks 'Edit Profile' button in the My Profile section header.

3. Form renders pre-populated with student.email and student.phone_number; registration number and full name display as read-only badges.

4. Student enters updated email and/or phone number; client clears field errors dynamically as the user types.

5. If student clicks 'Cancel', the form reverts to the read-only profile view without modifying data.

6. Student clicks 'Save Changes'; client executes validateProfileForm(). If invalid, error messages render inline.

7. On valid submission, client dispatches PUT /api/students/me with payload { email, phoneNumber } and Bearer JWT token.

8. Backend validates JWT, verifies student existence, validates input syntax, checks duplicate email uniqueness, and updates the database.

9. Backend responds with HTTP 200 OK containing success message and updated student record.

10. Frontend updates local student state, displays green success notification banner, and transitions back to the updated profile view.

2.5 Error and Data Handling

The existing authentication error handling and data security remain completely intact:

- Empty Fields: Blank submissions prompt 'Email address is required' or 'Phone number is required' immediately.
- Invalid Email: Malformed email strings prompt 'Email must follow a valid email format'.
- Invalid Phone: Non-numeric characters or lengths other than 10 digits prompt specific clear validation messages.
- Duplicate Email: Attempting to save an email owned by another student returns HTTP 409 Conflict with 'Email address is already registered'.
- Unauthenticated Access: Requests without valid JWT return HTTP 401 Unauthorized.
- Privilege Escalation Protection: Students attempting to access administrator PUT /api/students/:id receive HTTP 403 Forbidden.
- Database Resilience: Parameterized queries protect against SQL injection; unexpected database errors return HTTP 500 without crashing the server.

3. Testing & Verification

3.1 Testing Approach

The implementation was verified through automated API testing, database verification, and browser UI verification.

Testing Types:

- Functional testing
- Positive testing
- Validation testing
- Negative testing (invalid inputs & duplicate emails)
- UI testing (Edit Profile form, pre-filling, alerts)
- Theme compatibility (Light & Dark mode)
- Regression testing (Student login, profile retrieval, Admin CRUD)
- Security and Authentication verification

Environment: Development

Browser: [Chrome / Edge / Firefox]

Test Role: Administrator & Student

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Open Edit Profile form	Form opens; email and phone pre-filled with current values	Form opened with pre-filled inputs and static reg/name	Pass
TC-02	Cancel Edit Profile	Form closes; profile values remain unchanged	Form closed; original profile displayed	Pass
TC-03	Submit empty email	Validation error: 'Email address is required'; submission blocked	Inline validation error displayed; request blocked	Pass
TC-04	Submit invalid email format	Validation error: 'Email must follow a valid email format'	Inline validation error displayed; request blocked	Pass
TC-05	Submit empty phone number	Validation error: 'Phone number is required'; submission blocked	Inline validation error displayed; request blocked	Pass

Test ID	Test Case	Expected Result	Actual Result	Status
TC-06	Submit non-numeric phone	Validation error: 'Phone number must contain numeric characters only'	Inline validation error displayed; request blocked	Pass
TC-07	Submit phone with != 10 digits	Validation error: 'Phone number must contain exactly 10 digits'	Inline validation error displayed; request blocked	Pass
TC-08	Submit duplicate email of another student	HTTP 409 Conflict: 'Email address is already registered'	Status 409 returned; error banner displayed	Pass
TC-09	Submit valid email update	HTTP 200 OK; email updated in database and UI	Status 200 returned; updated email saved & displayed	Pass
TC-10	Submit valid phone update	HTTP 200 OK; phone updated in database and UI	Status 200 returned; updated phone saved & displayed	Pass
TC-11	Verify persistence via GET /api/students/me	Refreshed API call returns updated email and phone number	Status 200; updated profile persisted in MySQL	Pass
TC-12	Verify unauthenticated PUT /api/students/me	HTTP 401 Unauthorized returned	Status 401 returned for unauthenticated request	Pass
TC-13	Student attempt to access PUT /api/students/:id	HTTP 403 Forbidden returned; admin route protected	Status 403 returned; student cannot access admin route	Pass
TC-14	Verify administrator CRUD operations	Admin student list retrieval and management remain unaffected	Status 200; full administrator CRUD verified intact	Pass

3.3 Test Summary

Overall Test Result: Pass / Partially Passed / Failed

Test Result	Count
Total Test Cases	14
Passed	14
Failed	0
Pass Rate	100%

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	The student can open the Edit Profile form from My Profile.	UI verification in StudentDashboardPage (TC-01)	Met
AC-002	Current email and phone values are pre-filled.	Form state inspection upon opening (TC-01)	Met
AC-003	A valid email address can be saved successfully.	End-to-end update test (TC-09)	Met
AC-004	A valid phone number can be saved successfully.	End-to-end update test (TC-10)	Met
AC-005	Invalid email/phone input is rejected with a clear message.	Validation testing (TC-03, TC-04, TC-05, TC-06, TC-07, TC-08)	Met
AC-006	Updated values persist in the database.	Database inspection & GET /api/students/me (TC-11)	Met
AC-007	The updated values are displayed after saving.	UI profile card refreshed verification (TC-09, TC-10)	Met
AC-008	A student cannot update another student's profile.	Security test: identity derived from JWT userId (TC-12, TC-13)	Met
AC-009	Existing student login and profile retrieval continue to work.	Regression test: login & GET /api/students/me verified (TC-11)	Met
AC-010	Existing administrator CRUD functionality continues to work.	Regression test: GET /api/students & admin routes verified (TC-14)	Met

5. Evidence

A maximum of two relevant screenshots should be attached.

Evidence 01 – Edit Profile Form & Pre-populated Values

Related: AC-001, AC-002, FR-001, FR-002

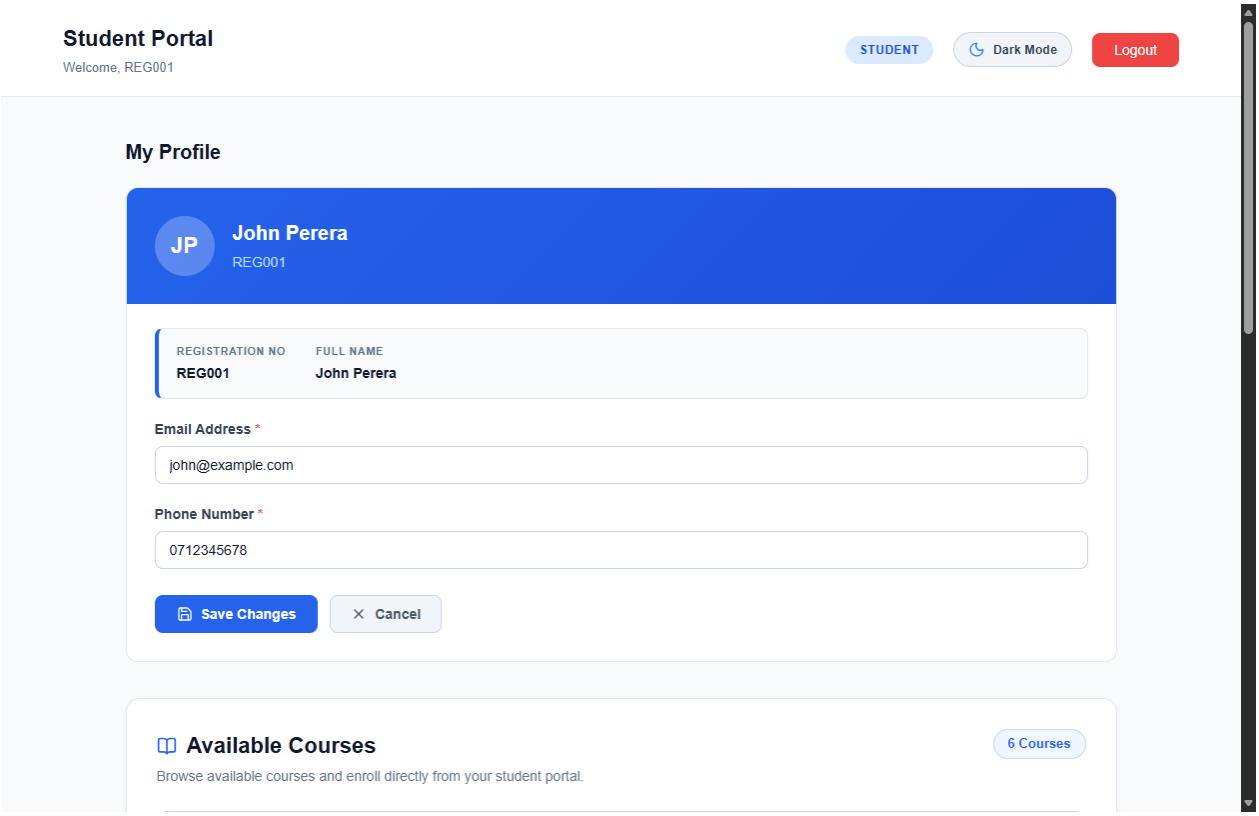


Figure 1: Student Dashboard showing the Edit Profile form with pre-filled email and phone number, read-only registration number and name, and Save/Cancel controls.

Evidence 02 – Successful Profile Update & Refreshed Profile Details

Related: AC-003, AC-004, AC-006, AC-007, FR-003, FR-004, FR-006, FR-007

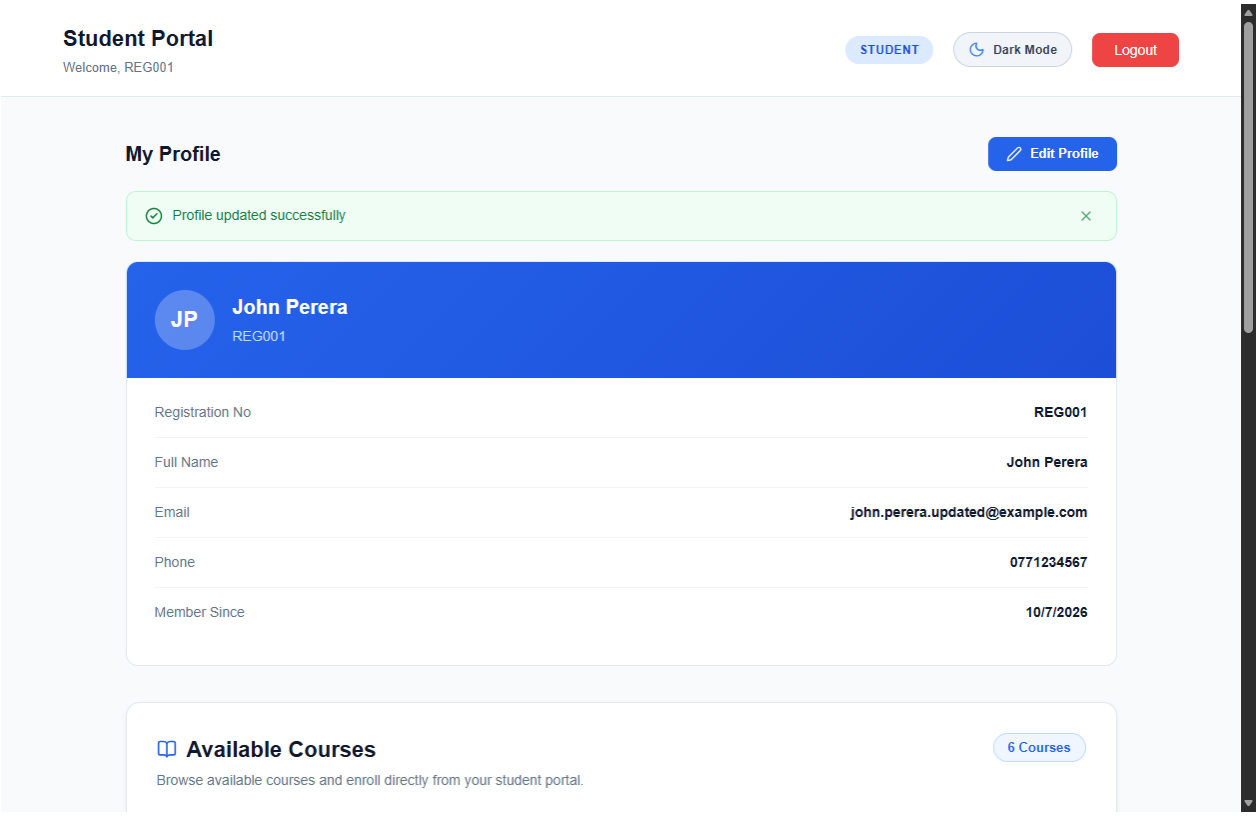


Figure 2: Student Dashboard displaying green success alert banner and refreshed profile details with updated email and phone number.

6. Final Result

Item	Result
Requirements	Met / Partially Met / Not Met
Implementation	Completed / Partially Completed
Testing	Passed / Partially Passed / Failed
Acceptance Criteria	All Met / Partially Met / Not Met
Existing CRUD / Auth	Verified / Not Verified
Final Status	Accepted / Pending / Rejected

6.1 Final Summary

CR-019 was successfully implemented as a complete full-stack student self-service profile editing enhancement for the Student Management System. The implementation enables enrolled students to securely update their own email address and phone number directly within their student portal dashboard without administrator assistance.

The solution strictly enforces client-side and server-side validation for RFC email format and 10-digit phone number structure, prevents duplicate email conflicts across student accounts, derives student identity exclusively from the authenticated JWT session token (preventing unauthorized cross-student updates), and refreshes profile details immediately upon persistence.

All existing student authentication, profile retrieval (GET /api/students/me), and administrator CRUD functionality (PUT /api/students/:id) remain completely intact and unaffected. The implementation satisfies 100% of defined functional and security requirements (FR-001 through FR-008, SEC-001 through SEC-006) and meets all ten acceptance criteria (AC-001 through AC-010) across all test scenarios.

6.2 Issues / Limitations

[State any remaining issues or limitations.]

If none:

No significant issues identified.

Student Name: MH Anush

Student ID: JFICT2416

Date: 29 October 2026

Signature: _____

~ END OF REPORT ~